Advanced Computer-Aided VLSI System Design
Final Project: End-to-End ResNet6 Accelerator with a Control Host and a External DRAM

TA: 蔡岳峰 f12943014@ntu.edu.tw **Due Tuesday, June. 3, 14:00**

Schedule

Review Slide Submission : **Tuesday, April 29, 12:00**

Project Review 1 : Tuesday, April 29

Project Review 2 : Tuesday, May 6

Due : **Tuesday, June 3, 14:00**

Presentation : Tuesday, June 10

Data Preparation

1. Decompress 1132_final.tar.gz with following command

```
tar -zxvf 1132_final.tar.gz
```

Folder	File	Description
00_TESTBED	TESTBED.sv	Testbench for designs
00_TESTBED	HOST.svp	Host Verilog file
00_TESTBED/PATTERN/IMAGES	image_*.bin	Binary data of images
00_TESTBED/PATTERN	categories.csv	Golden categories
00_TESTBED/PATTERN	confidences.csv	Golden confidences
00_TESTBED/PATTERN	model.dat	Soft link to model.dat
01_RTL	rtl_sim.f	File list for RTL simulation
01_RTL	01_run	RTL simulation script file
01_RTL	resacc.v	Your design
02_SYN	resacc.sdc	SDC file for synthesis
03_GATE	01_run	Gate simulation script file
03_GATE	gate_sim.f	File list for gate simulation
05_POST	post_sim.f	File list for post-layout simulation
05_POST	01_run	Post-layout simulation script file
05_POST	02_fsdb2vcd	FDSB to VCD script file
06_POWER	01_power	Power analysis script file
06_POWER	pt.tcl	Primetime script

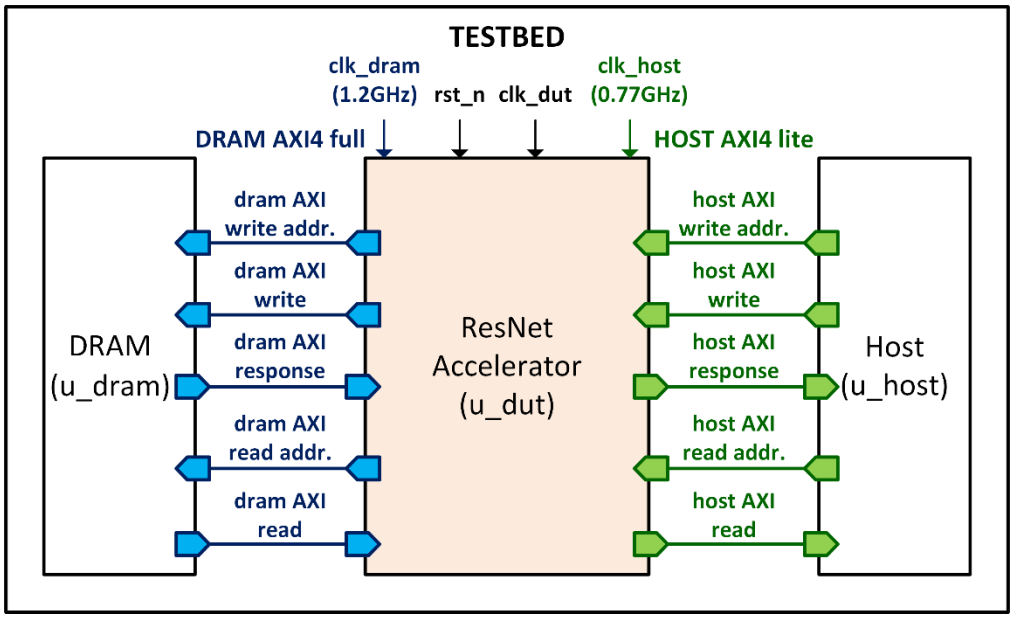
10_AXI_DRAM	AXI_DRAM.svp	DRAM Verilog file
98_ALGORITHM	environment.yml	Library setting file
98_ALGORITHM	model.dat	Example model.dat
98_ALGORITHM	resnet6.ipynb	Resnet6 Jupyter Notebook
98_ALGORITHM	resnet6.pth	Resnet6 parameter
98_ALGORITHM	resnet6_quantize.ipynb	Quantize Jupyter Notebook
/	design.spec	The spec of your design

All libraries needed for synthesis, simulation can be found in ADFP workstation. **Only worst-case library is used in this project.**

Introduction

In this project, you are asked to design an end-to-end **ResNet6 accelerator** for a Cifar-10 classifier. The input image and convolution parameters are stored in the external memory. The engine must fetch the image and parameters from the memory via the AXI4 full interface and then perform convolution on the image. The computation results are stored in the internal register of the accelerator and transmitted to the host through the AXI4 lite interface. There are three clock domains, which are for host signal, for memory, and for your design, respectively. The host will only check the error rate of the classification results.

Block Diagram



Specifications

1. Top module name: **resacc**

2. Input/output description:

Signal Name	I/O	Width	Simple Description
clk_dut	I	1	The system clock signal for the ResNet accelerator You can define the clock frequency (>= 100MHz)
clk_dram	I	1	The system clock signal for the signal from/to DRAM The frequency is fixed at **1.2GHz (clock period 0.834 ns)** The I/O should be synchronous to the negative edge of the clock.
clk_host	I	1	The system clock signal for the signal from/to host The frequency is fixed at **0.77GHz (clock period 1.3 ns)** The I/O should be synchronous to the negative edge of the clock.
rst_n	I	1	**Asynchronous** active low reset signal
host_axi_awaddr	I	8	Address of the first transfer in a transaction.
host_axi_awvalid	I	1	Write address request valid indicator.
host_axi_awready	O	1	Write address request ready indicator.
host_axi_wdata	I	32	Write data.
host_axi_wstrb	I	4	Indicator of which byte lanes of write data contain valid data.
host_axi_wvalid	I	1	Write data valid indicator.
host_axi_wready	O	1	Write data ready indicator
host_axi_bresp	O	2	Response for write channel **(Only 2'b00 allowed).**
host_axi_bvalid	O	1	Write response valid indicator.
host_axi_bready	I	1	Write response ready indicator.
host_axi_araddr	I	8	Address of the first transfer in a transaction.
host_axi_arvalid	I	1	Read address request valid indicator.
host_axi_arready	O	1	Read address request ready indicator.
host_axi_rdata	O	32	Read data.
host_axi_rresp	O	2	Response for read channel **(Only 2'b00 allowed).**
host_axi_rvalid	O	1	Read valid indicator.
host_axi_rready	I	1	Read ready indicator.
dram_axi_awid	O	8	ID of the write transaction.
dram_axi_awaddr	O	19	Address of the first transfer in a transaction.
dram_axi_awlen	O	8	Total number of transfers in a transaction
dram_axi_awsize	O	3	Maximum number of bytes in a transfer within a transaction.
dram_axi_awburst	O	2	Mode of the address increment.
dram_axi_awvalid	O	1	Write address request valid indicator.
dram_axi_awready	I	1	Write address request ready indicator.

dram_axi_wdata	O	256	Write data.
dram_axi_wstrb	O	32	Indicator of which byte lanes of write data contain valid data.
dram_axi_wlast	O	1	Indicator of the last data.
dram_axi_wvalid	0	1	Write data valid indicator.
dram_axi_wready	I	1	Write data ready indicator
dram_axi_bid	I	8	ID of the write response transaction.
dram_axi_bresp	I	2	Response for write channel **(Only 2'b00 allowed)**.
dram_axi_bvalid	I	1	Write response valid indicator.
dram_axi_bready	O	1	Write response ready indicator.
dram_axi_arid	O	8	ID of the read transaction.
dram_axi_araddr	O	19	Address of the first transfer in a transaction.
dram_axi_arlen	O	8	Total number of transfers in a transaction
dram_axi_arsize	O	3	Maximum number of bytes in a transfer within a transaction.
dram_axi_arburst	O	2	Mode of the address increment.
dram_axi_arvalid	0	1	Write address request valid indicator.
dram_axi_arready	I	1	Write address request ready indicator.
dram_axi_rid	I	8	ID of the read transaction.
dram_axi_rdata	I	256	Read data.
dram_axi_rresp	I	2	Response for read channel **(Only 2'b00 allowed).**
dram_axi_rlast	I	1	Indicator of the last data.
dram_axi_rvalid	I	1	Read valid indicator.
dram_axi_rready	O	1	Read ready indicator.

3. Bit true is not required, but the gate and post-layout simulation result should be the same as the RTL simulation
4. The error rate should be less than 0.37
5. The total latency should be less than $5x10^6$
6. The total area should be less than $5x10^5$
7. Only the worst-case library is used for synthesis.
8. The synthesis result of data type should NOT include any latch.
9. The slack for setup time and hold time should be non-negative.
10. Only the worst-case library is used in power analysis.
11. Optimization based on the specific test image is prohibited.
12. Model modification or re-train the weight is prohibited.
13. Only the macro layout is needed
14. VDD and VSS power rings should each be 2um wide, with 5 ring pairs required

15. The IR drop of static power analysis should be less than or equal to 5%
16. Dummy metal layers are not needed
17. Core filler must be added
18. GDSII of your design should merge the IP and SRAM layout
19. Ensure the APR DRC/LVS is completely error-free and the sign-off DRC/LVS check is not needed
20. **The total latency of 2 images should be less than the total latency of 64 images divided by 16 (latency of 2 images < latency of 64 images / 16)**

Design Description

1. Resnet 6

   The ResNet-6 model processes input images of size **3×32×32**. The first operation is a **zero bias 3×3 convolution with stride 2 and zero padding**, which reduces the spatial dimensions to **16×16** while increasing the number of feature channels to **16**. Two residual basic blocks are following, which contain two convolution layers for each. The first convolution within the block uses a **zero bias 3×3 kernel with stride 2 and zero padding**, reducing the **feature map to 8×8/4×4** while the output has **16/32 channels**. The second convolution in the block also uses a **zero bias 3×3 kernel but with stride 1 and zero padding**, preserving the size of the first convolution. A skip connection is applied from the block's input to its output, involving down-sampling to match dimensions before the addition. After the second residual block, the feature map has a shape of 32×4×4. A global average pooling layer is applied, which computes the average of each feature map, reducing the output to a 32-dimensional vector. This vector is then flattened and passed through a feedforward (fully connected) layer that outputs a 10-dimensional vector, corresponding to the scores for 10 different classes such as cat, ship, and airplane.

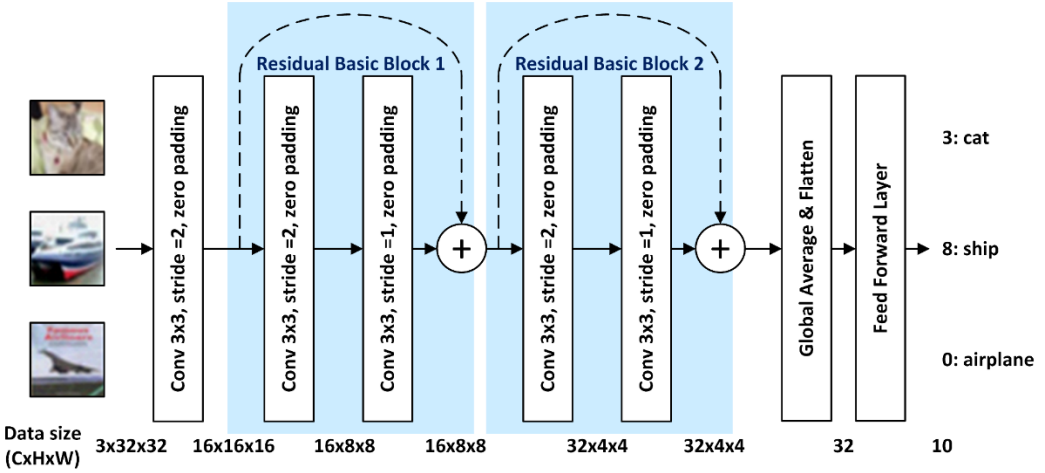


Fig. 1 ResNet 6 Architecture

2. Memory Mapping

The external memory has word size of 256-bit and can store up to 16384 words. To simplify notation, the *i*-th word in the memory is denoted as MEM[*i*].

4.1. Input Image

Input images are stored from 0x00000 to 0x0FFFF while each image occupies 0x00C00. The input images of the batch will be loaded to the external memory by the TESTBED at the beginning of the batch computation. Users don't need to handle this part.

4.2. Model Parameters

Model parameters are stored from 0x10000 to 0x3FFFF. Users must quantize and encode model parameters and form the memory file as a model.dat by themselves. The model.dat will be loaded to the external memory at the beginning of the whole simulation.

4.3. Temporary data

The temporary data can be stored from 0x40000 to 0x7FFFF. The accelerator can read or write the temporary data to this place during runtime.

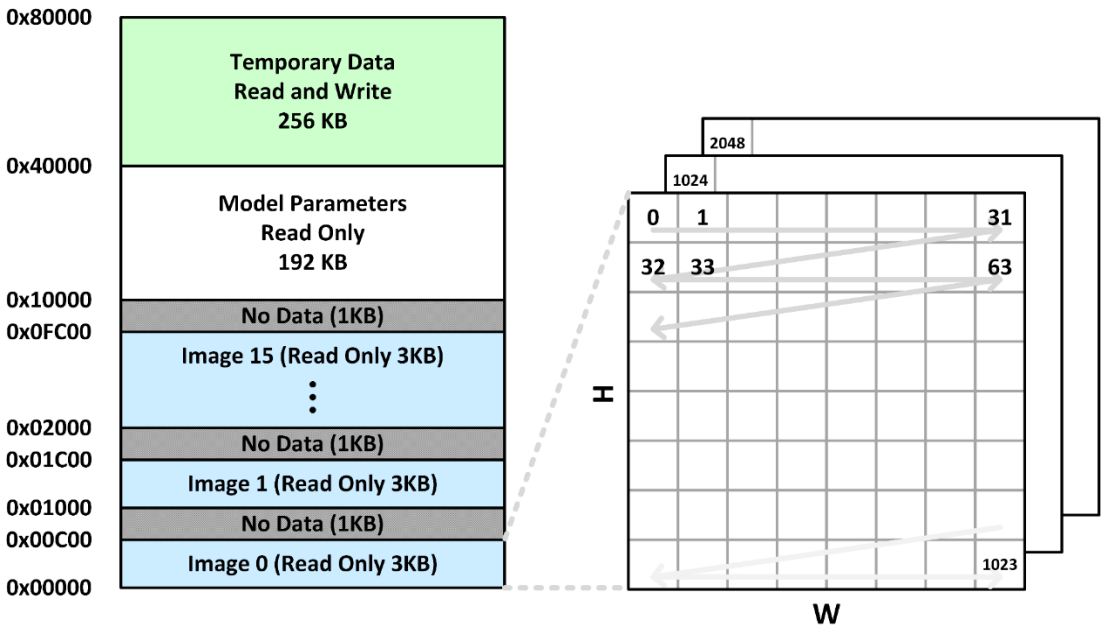


Fig. 2 Memory mapping of the external memory

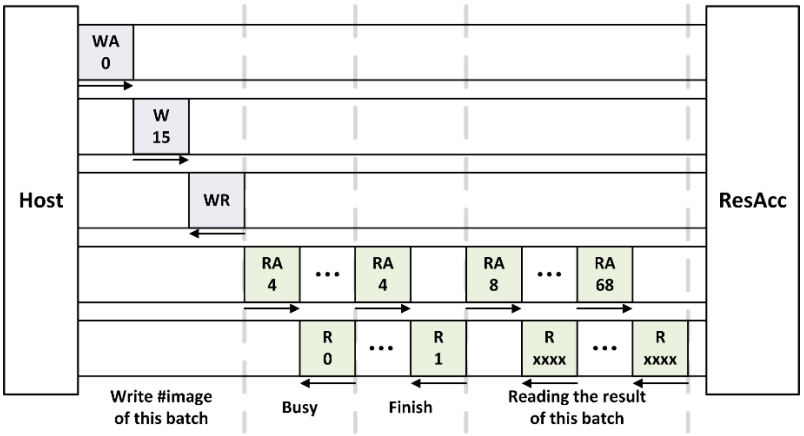
3. ResAcc Register

The register of the ResNet accelerator contains the number of images per batch, the finish signal of this batch, and the prediction results, while some of them are read-only and others are written-only. The data structure of ResAcc registers is shown in Fig. 3.

Addr.	Access	Data Content (32bit/data)	Data Note
0x44	Read Only	bit: 31-28 category bit: 27-0 confidence	The result of image 15
⋮	⋮	⋮	⋮
0x0C	Read Only	bit: 31-28 category bit: 27-0 confidence	The result of image 1
0x08	Read Only	bit: 31-28 category bit: 27-0 confidence	The result of image 0
0x04	Read Only	bit: 0 finish 1/busy 0	Finish & Busy indicator of this batch
0x00	Write Only	bit: 3-0 number of Image -1	The number of images of this batch (Ex: value 15 for 16 Images)

Fig. 3 ResAcc Register Structure

Timing Diagram



1. At the beginning of the simulation, the host will transfer how many images in this batch to the accelerator. The accelerator can start computing after receiving this package.
2. Then, the host will keep polling the data in address 4, which indicates whether the computation has finished.
3. Finally, the host will read the result from the register and check the categories of images in this batch.

Submission

1 Submission of Review Slides

1.1 Name the presentation slide to teamID_final_v*k*.pdf or teamID_final_v*k*.pptx (EX: team1_final_v1.pdf)

1.2 Both members of a team should hand it out to NTU COOL

1.3 Deadline: 2025/4/29 12:00:00 (UTC+8)

2 Submission in Workstation

2.1 Create a folder named **teamID_final** (in lowercase) (EX: team1_final) with the structure below:

```
team1_final/
├── 00_TESTBED/
│   └── model.dat
├── 01_RTL/
│   ├── resacc.v
│   ├── xxx.v (other verilog files, optional)
│   └── xxx.sv (other systemverilog files, optional)
│   └── rtl_sim.f
├── 02_SYN/
│   ├── resacc_syn.tcl
│   ├── resacc_syn.sdc
│   ├── resacc_syn.ddc
│   ├── resacc_syn.area
│   ├── resacc_syn.timing_min
│   └── resacc_syn.timing_max
├── 03_GATE/
│   ├── resacc_syn.v
│   ├── resacc_syn.sdf
│   └── gate_sim.f
├── 04_APR/
│   ├── final
│   ├── final.dat
│   └── resacc.gds
├── 05_POST/
│   ├── resacc_pr.v
│   ├── resacc_pr.sdf
│   └── post_sim.f
├── 06_POST/
│   └── resacc_pr.power
├── 07_UPF/ (optional)
│   ├── resacc.rtl.upf
│   ├── resacc.syn.upf
│   └── resacc.apr.upf
└── design.spec
```

2.2 07_UPF files are optional, and no submission of UPF files will not cause any penalty.

2.3 Pack the folder **teamID_final** (EX: team1_final) into a tar.gz file named **teamID_final.tar.gz** (EX: team1_final.tar.gz)

```
tar -zcvf teamID_final.tar.gz teamID_final
```

2.4 Put the tar.gz file at the root of both your ADFP accounts

TA will only check the latest version of your homework.

3 Submission in NTU COOL

3.1 Create a folder named teamID_final (in lowercase) (EX: team1_final) with the structure below:

```
team1_final/
├── 01_RTL/
│   ├── resacc.v
│   ├── xxx.v (other verilog files, optional)
│   └── xxx.sv (other systemverilog files, optional)
├── 98_ALGORITHM/
│   ├── dat_gen.py/dat_gen.ipynb
│   └── model.dat
├── report.pdf
└── design.spec
```

3.2 Compress the folder teamID_final (all lowercase) in a tar file named teamID_final_v*k*.tar.gz (EX: team1_final_v1.tar.gz)

```
tar -zcvf teamID_final_vk.tar.gz teamID_final
```

3.3 Both members of a team should hand it out to NTU COOL

Grading Policy

1. TA will use **01_run** at RTL, gate-level, and post-layout simulation
   - RTL simulation: 64 images and 2 images
   - Gate-level simulation: 2 images
   - Post-layout simulation: 2 images
2. The error rate of 64 images should be less than 0.37
3. The total latency (time) should be less than $5x10^6$ ns
4. The area after APR should be less than $5x10^5$ um^2
5. Implementation (40%)

	Score
RTL simulation	20%
Synthesis & Gate-level simulation	10%
APR & Post-layout simulation	10%

6. Performance Index

   The index for performance to be considered:

   $$\textbf{Performance} = \textbf{Area} \times \textbf{Time} \times \textbf{Power} \times (\textbf{Error Rate} - \textbf{0.1})$$

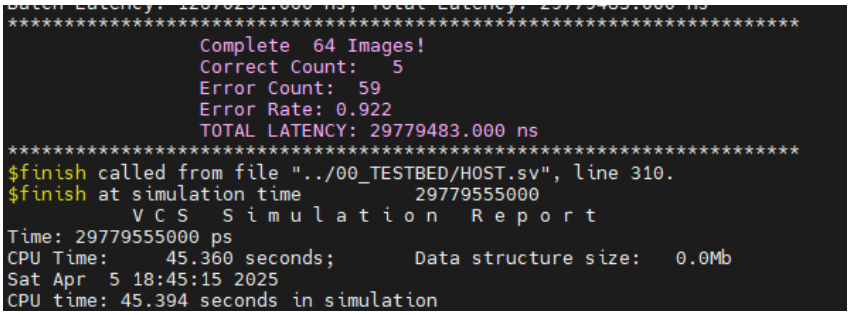
   Unit: power (mW), time (ns), area(um^2)

   Goal: Time < 10^6, Performance index < $5x10^{11}$

	Score
Goal	20%
Ranking (need passing goal)	30%

7. Precise explanation of the performance terms:
   - Error rate: the error rate will be reported from the 64-image RTL simulation

- Total latency (time): the total latency will be reported from the 64-image RTL simulation



- Area: total layout area in APR
- Power: total power in the 2-image post-layout simulation

8. Report and Review Presentation (10%)

	Score
Review Presentation	5%
Report	5%

Reference

[1] ResNet: Residual neural network – Wikipedia

[2] Cifar-10: CIFAR-10 and CIFAR-100 datasets

[2] Convolution: [ML 2021 (English version)] Lecture 9: Convolutional Neural Networks

[3] Convolution: ACVSD Midterm

[5] AXI protocol: AMBA AHB Protocol Specification